Historical Compensability-Guided Sharpness-Aware Training for Post-Training N:M Pruning

Anonymous CVPR submission

Abstract

Sharpness-aware training and second-order post-training pruning use different local descriptions of a network. The former allocates perturbations according to task gradients, whereas the latter reconstructs layer outputs after constrained weight deletions. We investigate whether conditional reconstructability can connect these stages. Historical Compensability-Guided SAM (HC-SAM) estimates damped activation inverse-Gram diagonals during dense training, accumulates their globally centered log scales, and uses the resulting history to redistribute a fixed global L_2 perturbation budget. The trained dense model is subsequently compressed by native SparseGPT, without changing its dynamic mask selection or compensation and without recovery training. We characterize the conditional single-deletion interpretation, geometric averaging, and budget competition underlying the method. These properties do not guarantee improved accuracy or solve the full joint N:M deletion problem. The framework separates the proposed training signal from the downstream compression algorithm, enabling its effect to be tested without changing native mask selection or compensation. The central empirical question is whether this training-side signal improves compatibility with an unchanged post-training compressor.

1. Introduction

Structured N:M sparsity constrains a fixed number of surviving weights within small groups, providing a regular representation for sparse execution [3]. Producing such a representation is not merely a matter of ranking individual magnitudes: deleting weights changes layer outputs, and surviving weights can partly compensate for those changes. Second-order post-training methods use calibration activations to model this reconstruction problem. Optimal Brain Compression and SparseGPT demonstrate how reconstruction geometry can guide deletion and compensation without retraining a dense model for each sparse support [4, 5]. Their effectiveness raises a complementary question: can the dense training procedure prepare a model for this specific kind of reconstruction?

Sharpness-aware minimization (SAM) trains against a local perturbation computed from the task gradient [9]. Its Euclidean perturbation and a structured pruning displacement differ in direction, support, and magnitude. A pruning displacement sets selected weights exactly to zero and is constrained by the N:M pattern; a SAM perturbation explores a small neighborhood and need not emphasize those coordinates. This mismatch motivates studying where the perturbation budget is allocated. However, keeping a fixed SAM radius cannot reproduce arbitrarily large finite deletions, and changing the direction alone should not be presented as resolving that discrepancy completely.

There is also a difference in the information available to the two stages. A task gradient measures local loss change at the current parameters. An activation reconstruction Gram describes how changes to several input-connected weights can offset each other in a layer output. In a fixed positive-definite quadratic model, the inverse-Gram diagonal controls the optimal cost of deleting one weight while allowing the other weights to adjust. Standard SAM does not explicitly use this conditional reconstruction information. Incorporating it into training offers a testable bridge between a dense optimization objective and a later second-order compressor, without requiring a sparse model to be trained at each step.

We study *Historical Compensability-Guided SAM* (HC-SAM). At each epoch boundary, a fixed set of training images supplies activation statistics for a damped inverse-Gram diagonal. We globally center its log scale, maintain an exponential history, and use the resulting positive scales to precondition the next epoch's SAM direction. A single normalization preserves the global L_2 perturbation radius. The final model is pruned with unchanged native SparseGPT. This isolates the proposed training signal from changes to the downstream selection or compensation algorithm. The method uses shared input-feature statistics, not independent second-order matrices for every output row.

Our contributions are threefold. First, we specify a causal training-to-pruning protocol that uses historical conditional reconstruction statistics to allocate a fixed perturbation budget. Second, we characterize the signal through its

OBS interpretation, relative geometric history, and budget-competition identities, while distinguishing these properties from unproven performance claims. Third, we formulate an explicit post-hoc perturbation-to-pruning alignment diagnostic to distinguish support-level association from task-level performance. Completed numerical tests support implementation feasibility. Accuracy gains, the value of history over an instantaneous estimate, and the proposed mechanism remain empirical questions; this draft does not claim an established state-of-the-art result.

2. Related Work

2.1. Sharpness-aware and preconditioned optimization

SAM uses a task-gradient perturbation to seek parameters with low neighborhood loss [9]. ASAM changes the perturbation geometry to address parameter-scale sensitivity [6], while SSAM restricts perturbation support [8]. Preconditioned SAM explicitly separates objective and constraint preconditioning [11]; HC-SAM's normalized direction is an instance of its objective-preconditioning form. GEAR-SAM uses historical gradient-energy statistics to redistribute blockwise perturbations while preserving a global radius [12]. These precedents rule out claiming that historical statistics, adaptive allocation, or fixed-radius preconditioning are novel in isolation. Our narrower hypothesis concerns the use of *activation inverse-Gram compensability history* to prepare a dense model for an unchanged post-training reconstruction method.

2.2. Compression-aware training

CrAM makes training sensitive to compression through compression-aware perturbations [1]. AdaSAP connects adaptive sharpness-aware perturbations to robust pruning [2]. These works already establish that dense optimization and compression need not be designed independently. HC-SAM does not claim to introduce compression-aware training. It instead uses a historical local reconstruction statistic without training a projected sparse network at each mini-batch. Its benefit over an instantaneous reconstruction statistic is an explicit ablation question. Original APBR is retained as a separate internal reference: it uses energy feedback, whereas HC-SAM's state update depends on inverse-Gram estimates and contains no such feedback term.

2.3. Second-order post-training pruning

Optimal Brain Compression develops layerwise reconstruction methods including ExactOBS [5]. SparseGPT shares curvature computations and uses sequential error compensation to make large-scale post-training pruning practical, including semi-structured patterns [4]. Wanda uses activation-aware magnitude scores without a compensating weight update [10]. ROSE studies feature ordering within SparseGPT-style pruning [7]. HC-SAM retains the native SparseGPT procedure rather than claiming its selection, ordering, or compensation mathematics as new. Training methods for $N{:}M$ sparsity also directly optimize a structurally constrained model [3]; our endpoint is instead produced after dense training. The distinction is consequential: HC-SAM incurs additional dense-training calibration, whereas SparseGPT alone can compress an already available model. Any practical argument must account for that extra cost.

3. Method

3.1. Problem formulation

We consider dense supervised training followed by post-training $N{:}M$ pruning: exactly N positions remain in each complete contiguous group of M eligible weights. Let $\mathcal{P}$ denote all trainable coordinates and $\mathcal{E} \subset \mathcal{P}$ the prunable coordinates. Dense optimization does not enforce a sparse support. After training, a fixed final checkpoint is compressed without gradient-based recovery. Our objective is to change the training trajectory so that an unchanged second-order compressor can better reconstruct the resulting network. This is a hypothesis about *training–compression compatibility*, rather than a new pruning solver.

HC-SAM augments the two-gradient SAM update with a persistent log-scale state. The state is estimated from training-side activation statistics at epoch boundaries and held fixed within the next epoch. It is neither optimized through the task loss nor updated from realized perturbation energy. The native SparseGPT endpoint does not read this state: any downstream effect must be mediated by the learned dense weights and activations.

The method can be understood as separating two questions. The task gradient tells us which direction currently changes the classification loss. Reconstruction statistics tell us which weight coordinates can be changed while allowing other coordinates to offset their effect on a layer output. HC-SAM retains the task gradient as the signed direction and uses the second signal only to change its relative scale. It does not substitute a reconstruction loss for the classification objective, delete weights during training, or prescribe the final sparse support.

Our design hypothesis is that perturbing conditionally replaceable coordinates during dense training can prepare a model for the changes later made by a reconstruction-based compressor. This is different from treating large inverse-Gram entries as a list of weights to remove. The final selection still depends on learned weight values and on compensation already performed by the compressor. We therefore evaluate the training procedure through its eventual compressed model rather than treating the geometry state itself

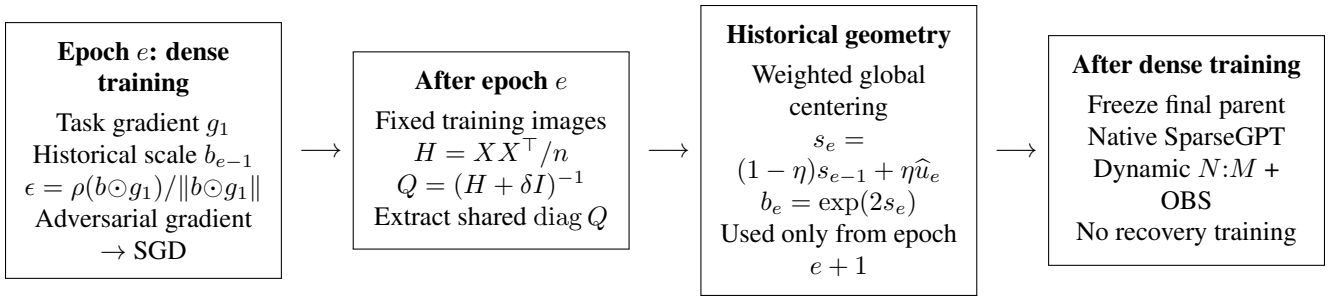


Figure 1. HC-SAM separates training-time geometry estimation from the final compressor. The first three boxes repeat across epochs; the last is executed only after dense training. Calibration uses training data, with BN statistics frozen. The final native mask is never fed back into training. This is a conceptual diagram, not an empirical result.

as a pruning result.

3.2. Conditional reconstruction geometry

For an eligible convolution ℓ, flatten its weights in output-row OIHW order as $W_\ell \in \mathbb{R}^{m_\ell \times d_\ell}$. Unfold calibration inputs into $X_{\ell,e} \in \mathbb{R}^{d_\ell \times n_{\ell,e}}$, where each column is an input patch. Input-channel and kernel coordinates follow exactly the same order as the flattened weights. For a row displacement Δw, the local physical output error is

$$\frac{\|X_{\ell,e}^\top \Delta w\|_2^2}{2n_{\ell,e}} = \frac{1}{2}\Delta w^\top H_{\ell,e}\Delta w, \quad H_{\ell,e} = \frac{X_{\ell,e}X_{\ell,e}^\top}{n_{\ell,e}}. \tag{1}$$

We use relative damping and its inverse,

$$B_{\ell,e} = H_{\ell,e} + \delta_{\ell,e} I, \quad \delta_{\ell,e} = \alpha \frac{\operatorname{tr} H_{\ell,e}}{d_\ell}, \quad Q_{\ell,e} = B_{\ell,e}^{-1}. \tag{2}$$

All subsequent properties assume $B_{\ell,e} \succ 0$. A failed factorization is an error, not an invitation to search for another damping value. The additional term $\delta\|\Delta w\|^2/2$ is part of the damped objective. Consequently, damped costs and undamped physical reconstruction MSE must be distinguished.

The diagonal Q_{jj} measures conditional freedom within this quadratic model. With the current row w, deleting one coordinate while allowing the remaining coordinates to adjust costs $w_j^2/(2Q_{jj})$ [5]. Thus, a large Q_{jj} means lower cost *at fixed* w_j. It does not imply small absolute cost, and it does not identify the final multi-coordinate pruning support. Importantly, H is an activation reconstruction Gram, not the cross-entropy Hessian or a full-network curvature matrix.

The distinction between a Gram diagonal and an inverse-Gram diagonal matters. The former measures the energy of an input feature considered by itself. The latter also reflects relationships with the other features through the matrix inversion. For example, changing a coefficient connected to a redundant feature may be partly offset by changing coefficients connected to correlated features. Conditional compensability captures this possibility within the damped local model. It does not assert that the feature is unimportant for classification, nor that several individually replaceable features can all be removed safely together.

Only the inverse diagonal is retained as the training signal. This choice provides one scale per input coordinate while avoiding a distinct curvature matrix for every output row. Nevertheless, computing that diagonal still requires a dense layerwise factorization and solve. The method reduces the state used by the optimizer; it does not make curvature estimation free.

3.3. Historical compensability estimation

Write $i = (\ell, o, j)$ for a weight coordinate and $D = |\mathcal{E}| = \sum_\ell m_\ell d_\ell$. Broadcast each inverse-Gram diagonal across output rows:

$$u_{i,e} = \tfrac{1}{2}\log Q_{\ell,e,jj}, \tag{3}$$

$$\bar{u}_e = \frac{1}{2D}\sum_\ell m_\ell \sum_j \log Q_{\ell,e,jj}, \qquad \widehat{u}_{i,e} = u_{i,e} - \bar{u}_e. \tag{4}$$

This is one global, coordinate-weighted centering operation. Averaging layer means equally would define a different algorithm. Starting from $s_{i,-1} = 0$, update only after completing epoch e:

$$s_{i,e} = (1-\eta)s_{i,e-1} + \eta\widehat{u}_{i,e}. \tag{5}$$

Epoch $e+1$ uses s_e; epoch 0 uses the zero state and therefore the ordinary SAM direction. The coefficient η controls memory rather than an energy-feedback rate. With identical zero initialization, $s_{\ell,o,j,e}$ is identical across output rows o. HC-SAM therefore learns a layerwise input-feature geometry broadcast to weights, *not* independently learned output-specific scales. The perturbations can still differ across rows because their task gradients differ.

The log-domain update stabilizes relative scale estimates across epochs. A constant common rescaling of every Q

is removed by global centering. Layer-specific rescaling is not removed and must not be introduced through inconsistent Gram conventions. Noneligible coordinates retain unit scale, so centering also fixes the relative budget between eligible and noneligible parameters.

History is introduced because the reconstruction geometry changes with the dense model. An estimate from one epoch describes that checkpoint and its calibration activations; it is not a permanent property of a weight coordinate. The moving state blends recent estimates instead of replacing the perturbation geometry abruptly at every refresh. Whether this memory is useful or merely delays adaptation is an empirical question, which motivates the Direct-Q control. The logarithmic representation makes the blend multiplicative in scale and keeps all resulting direction weights positive.

The timing is deliberately conservative. Statistics measured after epoch e affect only epoch $e+1$ and later. Within an epoch, every mini-batch uses the same committed geometry, so the optimizer does not need a new factorization for each update. The first epoch begins with no reconstruction preference and follows the ordinary SAM direction. Historical guidance emerges only after observing the trained network's own activations.

3.4. Competitive fixed-budget perturbation

For a mini-batch in epoch e, compute the center task gradient $g_1 = \nabla_w \mathcal{L}(w)$. Define

$$b_i = \begin{cases} \exp(2s_{i,e-1}) & i \in \mathcal{E}, \\ 1 & i \notin \mathcal{E}, \end{cases} \qquad z = b \odot g_1, \quad \epsilon = \rho \frac{z}{\|z\|_2}. \tag{6}$$

The denominator includes *all* trainable coordinates with valid gradients, including parameters excluded from pruning. If $z = 0$, set $\epsilon = 0$ and record that step. Evaluate $g_2 = \nabla_w \mathcal{L}(w + \epsilon)$, restore the center weights, and apply the original SGD update using g_2. Geometry is detached; no gradient is propagated through the calibration calculation or the perturbation construction. Batch-normalization running statistics are updated on the center pass only, and calibration uses evaluation mode without changing buffers.

Equation (6) redistributes a fixed Euclidean budget. It is generally *not* the maximizer of $g_1^\top \epsilon$ over the Euclidean ball; that maximizer is the ordinary SAM direction. It can instead be viewed as maximizing $(b \odot g_1)^\top \epsilon$ over that ball, an objective-preconditioning instance of the preconditioned SAM framework [11]. The proposed ingredient is the historical reconstruction-based choice of b, not the general normalized preconditioning formula.

At a fixed nonzero gradient, increasing one coordinate's scale gives that coordinate a larger share of the perturbation energy. Because the total radius is unchanged, this share is taken from other coordinates rather than added to the overall budget. The resulting probe asks the classifier to remain useful under a different distribution of local parameter changes. The second task gradient then updates the restored dense model in the usual way. No second-order term is added to the supervised loss, and no gradient through Q is needed.

Giving more budget to a compensable coordinate is a deliberate choice, not a rule dictated by OBS. Our hypothesis emphasizes directions that a later compressor may alter; an alternative policy could instead emphasize coordinates that are difficult to compensate. The singleton cost identity does not decide which policy will train the better classifier. HC-SAM fixes the former policy and tests it, while retaining task-gradient dependence: even a large geometry scale produces no perturbation at a coordinate whose current gradient is zero.

3.5. Native SparseGPT reconstruction

After the last epoch, freeze the dense checkpoint and collect the prescribed training-side calibration activations. The main endpoint runs native SparseGPT selection and OBS compensation [4]. At each $N{:}M$ group, its implementation ranks the current individual weights with a factor-based score, selects the required deletions, and propagates errors in its prescribed column order. Later groups see weights modified by earlier compensation. We do not replace this procedure by a globally precomputed mask, enumerate joint deletion sets, or introduce geometry into its score.

Two auxiliary endpoints isolate different operations. *Final-Q mask-only* ranks $|w_{o,j}|/\sqrt{Q_{jj}}$ on the frozen dense weights and directly zeros selected positions. *Final-Q fixed-mask OBS* starts again from the original dense weights and compensates using that externally fixed mask. The latter must not start from an already zeroed model, which would lose the original deletion displacement. Neither auxiliary endpoint is called native SparseGPT. All endpoints retain the same original BN buffers and use zero recovery epochs.

Keeping the native compressor unchanged is important for interpretation. HC-SAM does not seek a favorable result by modifying both the trained model and its final reconstruction algorithm simultaneously. Instead, its learned weights and activations are presented to the existing compressor under a frozen protocol. If the resulting sparse model performs better, the observation concerns the training-to-compression pipeline; additional controls are still needed to attribute that difference specifically to the history mechanism.

3.6. Training-to-pruning algorithm

Algorithm 1 states the time ordering explicitly. The final refresh estimates the auxiliary Final-Q target; it cannot retroactively change the last training epoch. The main native endpoint recomputes its own reconstruction factors un-

Algorithm 1 HC-SAM with a native post-training endpoint

Require: Dense initialization w_0, train data, calibration IDs, T, ρ, η, α, eligibility $\mathcal{E}$
Initialize $s_{-1} = 0$; initialize SGD and scheduler
for $e = 0, \dots, T-1$ **do**
  for training mini-batch $\mathcal{B}$ **do**
    $g_1 \leftarrow \nabla \mathcal{L}_{\mathcal{B}}(w)$ at center weights
    $\epsilon \leftarrow$ Eq. (6) using s_{e-1}
    Save center; add ϵ; freeze BN running updates
    $g_2 \leftarrow \nabla \mathcal{L}_{\mathcal{B}}(w+\epsilon)$
    Restore center even on an exception
    Update restored w by SGD with g_2
  end for
  for each eligible layer ℓ, serially **do**
    Stream fixed calibration batches in evaluation mode
    Accumulate $H_{\ell,e}$; factor $B_{\ell,e}$
    Extract diag $Q_{\ell,e}$; release layer matrices
  end for
  Compute weighted global target by Eq. (4)
  Update s_e by Eq. (5); advance scheduler
  Save weights, geometry, optimizer, RNG, and evidence
end for
Freeze final dense checkpoint and calibration protocol
Run native SparseGPT $N{:}M$ selection and compensation
Audit masks, zero entries, finite values; freeze hashes
Evaluate fixed dense and sparse endpoints once

der the frozen compressor convention.

4. Theoretical Analysis

The following results characterize the proposed signal and budget allocation. They do not establish superior task accuracy, convergence, or generalization. Proofs are given in Appendix A.

4.1. Single-coordinate OBS cost

Proposition 1 (Conditional deletion cost)**.** *Let $B \succ 0$, $Q = B^{-1}$, and w be a fixed active weight row. If all coordinates except a designated deletion j are free to adjust, then*

$$\min_{\Delta w:\, \Delta w_j = -w_j} \frac{1}{2} \Delta w^\top B \Delta w = \frac{w_j^2}{2Q_{jj}}, \qquad \Delta w^* = -\frac{w_j}{Q_{jj}} Q_{:j}. \tag{7}$$

This standard OBS result supplies the meaning of conditional compensability; it is not our contribution [5]. A corresponding group cost is $\frac{1}{2} w_S^\top (Q_{SS})^{-1} w_S$ when the other coordinates remain free. Generally it is not the sum of singleton costs. After earlier deletions or restrictions, the relevant inverse is that of the current free-coordinate quadratic. A diagonal estimated before training updates does not encode all such later conditions.

4.2. Historical geometric averaging

Define $G_k = \exp(D^{-1} \sum_{i \in \mathcal{E}} \log Q_{k,ii})$, with the output-row multiplicities from Eq. (4). Here $Q_{k,ii}$ denotes the shared diagonal value mapped to coordinate i.

Proposition 2 (Relative geometric history)**.** *For the zero initialization and fixed $\eta \in [0,1]$, the eligible scale satisfies*

$$b_{i,e} = \prod_{k=0}^{e} \left(\frac{Q_{k,ii}}{G_k} \right)^{\eta(1-\eta)^{e-k}}, \qquad \sum_{i \in \mathcal{E}} s_{i,e} = 0. \tag{8}$$

The observed exponents sum to $1 - (1-\eta)^{e+1}$; the remaining mass belongs to the initial unit scale. Thus early epochs shrink toward isotropy, without EMA bias correction. Setting $\eta = 0$ gives the standard SAM direction, while $\eta = 1$ gives Direct-Q with the *same global centering and unit noneligible scales*. The history averages log inverse-Gram scales, not realized perturbation vectors or their energies.

4.3. Fixed radius and budget competition

Proposition 3 (Fixed-budget redistribution)**.** *For nonzero $b \odot g$ and positive b, Eq. (6) has $\|\epsilon\|_2 = \rho$. Define $p_i = \epsilon_i^2 / \rho^2$. Then $\sum_i p_i = 1$, and, holding g and other scales fixed,*

$$\frac{\partial p_i}{\partial \log b_i} = 2p_i(1-p_i), \tag{9}$$

$$\frac{\partial p_j}{\partial \log b_i} = -2p_i p_j \quad (j \neq i). \tag{10}$$

The derivatives describe competition in a fixed budget. Increasing one scale increases its energy share unless its gradient is zero or it already receives all the energy. They are partial derivatives with respect to an unconstrained scale; a centered update changing several scales has the corresponding combined derivative. They do not imply that every coordinate with a large Q_{ii} receives a large realized share: g_i^2 matters as well.

4.4. What these properties do not establish

Three distinctions are essential. First, high conditional compensability, low actual deletion cost, and membership in the final SparseGPT deletion set are different concepts. Second, a constant global L_2 radius does not reproduce the magnitude or direction of a finite deletion displacement. Third, more perturbation energy on a future deleted support is a diagnostic association, not a sufficient condition for better classification.

For example, a coordinate with a large inverse diagonal but a zero task gradient receives no perturbation in Eq. (6). A pair of highly correlated features may each be individually replaceable while being costly to remove together. The history is also delayed: it summarizes several past models, whereas the final compressor sees a different model. These are reasons to test the proposal, rather than consequences resolved by the propositions. The log-EMA is a specified statistical mechanism, not an optimal solution derived from the task loss.

5. Experiments

6. Conclusion and Limitations

HC-SAM studies whether conditional reconstruction information can connect dense sharpness-aware training with post-training N:M compression. It accumulates globally centered inverse-Gram log scales and uses their history to redistribute a fixed perturbation budget, leaving the final native SparseGPT algorithm unchanged. The mathematical properties establish a well-defined signal and allocation mechanism; completed numerical tests establish implementation feasibility. They do not establish better sparse accuracy.

The current study is limited by an unfinished single-seed endpoint, missing Direct-Q controls, unavailable coordinatewise alignment logs, and measurable calibration overhead. Shared input-feature diagonals do not capture the complete interactions of jointly deleted weights. A positive final result would motivate matched multi-seed tests of history and reconstruction alignment; a negative result would constrain this proposed bridge rather than invalidate second-order pruning. Broader architectures, sparsity patterns, and actual sparse-kernel speedups remain outside the present evidence.

LLM Usage Statement

An OpenAI Codex assistant was used to organize the draft, inspect implementation and experiment records, assist with mathematical exposition and literature discovery, and generate LaTeX and audit scripts. Human authors must verify the scientific claims, citations, and final manuscript. No model-generated experimental observations are reported as measurements.

References

[1] Alexandra Peste, Adrian Vladu, Eldar Kurtic, Christoph H. Lampert, and Dan Alistarh. CrAM: A Compression-Aware Minimizer, 2022. Version 2207.14200v4. 2

[2] Anna Bair, Hongxu Yin, Maying Shen, Pavlo Molchanov, and Jose Alvarez. Adaptive Sharpness-Aware Pruning for Robust Sparse Networks, 2023. Version 2306.14306v2. 2

[3] Aojun Zhou, Yukun Ma, Junnan Zhu, Jianbo Liu, Zhijie Zhang, Kun Yuan, Wenxiu Sun, and Hongsheng Li. Learning N:M Fine-grained Structured Sparse Neural Networks From Scratch, 2021. Version 2102.04010v2. 1, 2

[4] Elias Frantar and Dan Alistarh. SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot, 2023. Version 2301.00774v3. 1, 2, 4

[5] Elias Frantar, Sidak Pal Singh, and Dan Alistarh. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning, 2022. Version 2208.11580v2. 1, 2, 3, 5

[6] Jungmin Kwon, Jeongseop Kim, Hyunseo Park, and In Kwon Choi. ASAM: Adaptive Sharpness-Aware Minimization for Scale-Invariant Learning of Deep Neural Networks, 2021. Version 2102.11600v3. 2

[7] Mingluo Su and Huan Wang. ROSE: Reordered SparseGPT for More Accurate One-Shot Large Language Models Pruning, 2026. Version 2603.05878v1. 2

[8] Peng Mi, Li Shen, Tianhe Ren, Yiyi Zhou, Xiaoshuai Sun, Rongrong Ji, and Dacheng Tao. Make Sharpness-Aware Minimization Stronger: A Sparsified Perturbation Approach, 2022. Version 2210.05177v2. 2

[9] Pierre Foret, Ariel Kleiner, Hossein Mobahi, and Behnam Neyshabur. Sharpness-Aware Minimization for Efficiently Improving Generalization, 2020. Version 2010.01412v3. 1, 2

[10] Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A Simple and Effective Pruning Approach for Large Language Models, 2023.

[11] Yilang Zhang, Bingcong Li, and Georgios B. Giannakis. Preconditioned Sharpness-Aware Minimization: Unifying Analysis and a Novel Learning Algorithm, 2025. Version 2501.06603v1. 2, 4

[12] Zhen Huang, Jiaxin Deng, and Junbiao Pang. Gradient-Energy Guided Block-Wise Perturbations for Sharpness-Aware Minimization, 2026. Version 2607.18306v1. 2

A. Proofs and Counterexample Boundaries

A.1. Conditional deletion cost

For Proposition 1, introduce a multiplier γ for $e_j^\top \Delta w + w_j = 0$. Stationarity of $\frac{1}{2}\Delta w^\top B \Delta w + \gamma(e_j^\top \Delta w + w_j)$ gives $\Delta w = -\gamma Q e_j$. The constraint implies $\gamma = w_j/Q_{jj}$ because positive definiteness ensures $Q_{jj} > 0$. Substitution gives

$$\tfrac{1}{2}\Delta w^{*\top} B \Delta w^* = \tfrac{1}{2}\frac{w_j^2}{Q_{jj}^2} e_j^\top Q B Q e_j = \frac{w_j^2}{2Q_{jj}}. \tag{11}$$

Strict convexity makes the feasible minimizer unique. For a deletion set S, replace e_j by the column selector E_S to obtain

$$\Delta w^* = -Q E_S (Q_{SS})^{-1} w_S, \qquad C_S = \tfrac{1}{2} w_S^\top (Q_{SS})^{-1} w_S. \tag{12}$$

This proves a local constrained quadratic identity, not a globally optimal sequence of structured deletions. It also shows why the diagonal is insufficient for group interactions. For instance, let $Q_{SS} = \left[\begin{smallmatrix} 1 & r \\ r & 1 \end{smallmatrix}\right]$ with $|r| < 1$ and $w_S = (1,1)^\top$. Each singleton cost is $1/2$, but the pair costs $1/(1+r)$, which differs from their sum unless $r = 0$. With $w_S = (1,-1)^\top$, the pair instead costs $1/(1-r)$. The same diagonal permits opposite interaction effects.

If a prior set is fixed, the free-coordinate objective must be formed before applying the identity. Inverting the appropriate free Hessian is generally not equivalent to selecting a principal block of the original inverse. SparseGPT's factor-based sequential approximation has its own active-subproblem interpretation; our training statistic should not be conflated with every diagonal used during that process.

A.2. Weighted centering and log history

Equation (4) counts each feature m_ℓ times because the shared statistic affects that many weight coordinates. Thus $\sum_{i\in\mathcal{E}} \widehat{u}_{i,e} = 0$. The zero initial state and Eq. (5) imply by induction that $\sum_i s_{i,e} = 0$. Expanding the recursion yields

$$2s_{i,e} = \eta \sum_{k=0}^{e} (1-\eta)^{e-k} \big(\log Q_{k,ii} - \log G_k\big). \tag{13}$$

Exponentiation proves Eq. (8). The omitted initialization term is $(1-\eta)^{e+1} 2s_{i,-1} = 0$, equivalently a unit-scale factor with that weight. This is a geometric average of relative inverse-diagonal values and the initial unit value. It is not an arithmetic mean of Q, and it is not an average of realized perturbation energy. The limits $\eta = 0$ and $\eta = 1$ follow directly from the recurrence.

Multiplying every layer's Q by the same positive constant adds a common shift to u which centering removes. Different constants for different layers do not cancel. Because noneligible coordinates use $b = 1$, replacing global centering with layerwise centering changes the physical direction even if within-layer ratios remain the same. DirectQ is therefore defined with the exact same conventions, not merely as an informal "multiply gradients by Q" rule.

A.3. Radius and energy derivatives

Let $Z = \sum_j b_j^2 g_j^2 > 0$. Then $\epsilon_i^2 = \rho^2 b_i^2 g_i^2 / Z$, so $\|\epsilon\|_2^2 = \rho^2$ and $p_i = b_i^2 g_i^2 / Z$. Setting $h_i = \log b_i$ and using the quotient rule,

$$\partial_{h_i} p_i = \frac{2b_i^2 g_i^2 Z - 2(b_i^2 g_i^2)^2}{Z^2} = 2p_i(1-p_i), \tag{14}$$

$$\partial_{h_i} p_j = -\frac{2b_i^2 g_i^2 b_j^2 g_j^2}{Z^2} = -2p_i p_j \quad (j \neq i). \tag{15}$$

The sum of these derivatives is zero. This establishes redistribution, not an increase in total energy. If all valid gradients vanish, the defined zero perturbation has norm zero and the proportions are undefined. The proof is in exact arithmetic; the implementation separately measures the norm after casting the perturbation to parameter precision.

The fixed-radius construction is not standard SAM's Euclidean inner maximizer unless $b \odot g$ is parallel to g. For $g = (1,1)$ and $b = (2,1)$ it is not. It does maximize the linear surrogate $(b \odot g)^\top \epsilon$ over the same ball. No claim that this surrogate is the first-order expansion of the original loss is needed or made.

A.4. Random-mask alignment and streaming estimators

Let $D_{\mathrm{GPT}} \subset \mathcal{E}$ be the frozen final native deletion set. For a training step with nonzero eligible energy, define the post-hoc support alignment

$$\mathcal{A}_t^{\mathcal{E}} = \frac{\sum_{i\in D_{\mathrm{GPT}}} \epsilon_{i,t}^2}{\sum_{i\in\mathcal{E}} \epsilon_{i,t}^2}. \tag{16}$$

The final mask is not available to the training algorithm and is used only retrospectively. This diagnostic measures energy on a support, not directional agreement or classification performance. Condition on a fixed energy vector $h_i = \epsilon_i^2$ and choose uniformly two deletions in every complete group of four, independently of h. Each coordinate has deletion probability $1/2$, so linearity of expectation gives

$$\mathbb{E}_D\left[\frac{\sum_{i\in D} h_i}{\sum_{i\in\mathcal{E}} h_i}\right] = \frac{\sum_{i\in\mathcal{E}} (1/2) h_i}{\sum_{i\in\mathcal{E}} h_i} = \tfrac{1}{2}. \tag{17}$$

The denominator must be nonzero. No independence between different mask entries is required. Dense tails, if included in $\mathcal{E}$ but never deleted, change this reference to one half of the energy on complete groups divided by total eligible energy. The current 19-layer setup has no incomplete four-element groups; general implementations must check this condition rather than assume it.

To recover pooled alignment, store $S_i = \sum_t \epsilon_{i,t}^2$ and compute $\sum_D S_i / \sum_{\mathcal{E}} S_i$ after freezing the mask. To recover the arithmetic mean of stepwise alignment, instead store

$$R_i = \sum_{t: Z_t^{\mathcal{E}} > 0} \frac{\epsilon_{i,t}^2}{Z_t^{\mathcal{E}}}, \qquad Z_t^{\mathcal{E}} = \sum_{j \in \mathcal{E}} \epsilon_{j,t}^2, \tag{18}$$

and divide $\sum_D R_i$ by the number of nonzero eligible-energy steps. These statistics coincide only under additional conditions such as constant $Z_t^{\mathcal{E}}$. Step counts, zero-step handling, and aggregation scope must be logged. Layerwise totals alone cannot determine either numerator for an arbitrary future within-layer mask.

B. Implementation and Evaluation Protocol

Frozen training. The exploratory configuration uses 50,000 training images, 10,000 test images, batch size 128, 391 batches per epoch, and 200 epochs (78,200 optimizer steps). The optimizer is SGD with initial learning rate 0.1, momentum 0.9, weight decay 5×10^{-4}, and cosine scheduling. The perturbation radius is $\rho = 0.05$, historical coefficient $\eta = 0.1$, and relative damping coefficient $\alpha = 0.01$. The full specification is versioned separately. No validation split or test-based model selection is introduced. The fixed last parent has zero-based epoch index 199. Smoke checkpoints are temporary and are not training initializations.

The eligible set contains 11,157,504 coordinates in 19 internal convolutions, including projection shortcuts. Strict 2:4 masks remove 5,578,752 positions. The first convolution, final linear layer, BN affine parameters, and biases are excluded from pruning but not from the global perturbation normalization. We record coordinate mappings and keep-masks as Boolean tensors. Every complete group retains exactly two positions; an incomplete tail, if encountered in another architecture, remains dense. Deleted weights must be exactly zero, and all output tensors must be finite.

Calibration and precision. The calibration index list contains 1,024 fixed training IDs drawn with the frozen generator and seed. Calibration has no augmentation, uses batch size 128, and streams one layer at a time. Convolutional unfold columns are accumulated into a Gram normalized by the total number of patches, not the number of batches. This protocol repeats the dataset traversal for each layer; it does not cache all unfolded activations. Gram arithmetic is FP32, while geometry is FP64. AMP and TF32 are disabled in the frozen configuration. CPU/GPU numerical tests compare explicitly stated tensors and precisions, not nominal dtype labels alone.

If $B = LL^\top$ is its Cholesky decomposition, $Q = L^{-\top}L^{-1}$ and $Q_{jj} = \sum_k (L^{-1})_{kj}^2$. This extracts the required diagonal without treating a Cholesky diagonal as an inverse-Hessian diagonal. It still requires a dense triangular solve, and therefore does not eliminate the cubic decomposition/solve cost. The calculation has $O(d^2 n)$ Gram cost, $O(d^3)$ factor/solve cost, and $O(d^2)$ working matrix memory per layer. Geometry can be represented by $O(\sum_\ell d_\ell)$ shared values, although the current implementation may retain broadcast tensors for parameter mapping. A memory bound on one matrix is not a total-process bound.

Gram conventions and dead features. HC training uses $H = XX^\top/n$. The audited native SparseGPT adapter uses $2XX^\top/n$, its historical zero-diagonal replacement rule, relative damping, block size 128, and FP32 compensation. In the absence of special dead-feature replacement, doubling both Gram and damping halves Q; the common log shift cancels under global centering. Dead-feature handling can break this simple scaling equivalence. The HC path rejects an invalid damped system rather than silently borrowing the native replacement rule. Main native evaluation preserves its own frozen convention; auxiliary static masks and fixed-mask compensation record theirs separately.

BN and checkpoint semantics. The center pass uses the ordinary training BN update. The adversarial pass prevents a second running-statistics update. Calibration preserves all BN buffers and restores model mode. Resume validation includes model weights, historical geometry, optimizer momentum, scheduler state, and Python, NumPy, CPU, and CUDA RNG state. Configuration and source hashes accompany checkpoints. Exceptions restore perturbed parameters before propagating the error; numerical failure does not silently switch to SAM.

Evaluation ordering. The final dense parent is immutable. The native, mask-only, and fixed-mask endpoints are independently derived from its original weights, sharing its BN buffers. The final calibration activations are collected from that dense parent, not from a model incrementally pruned by a preceding endpoint. Layer audit records include masks, reconstruction diagnostics, compensation statistics, and hashes. All endpoint checkpoints and the pre-test manifest are finalized before any test-loader access. Test accuracy and mean cross-entropy are evaluated once per frozen endpoint. The evaluation scripts contain no best-epoch selection, recovery optimizer, or damping search.

Reproducibility identities. The implementation is independently versioned as HQ-APBR-V1. The formal training source starts from commit `90499e9`, with a separate baseline mathematical implementation and immutable configuration digest in the accompanying evidence manifest. The final paper import records the complete source commit, configuration hash, dense parent hash, sparse checkpoint

and mask hashes, calibration IDs, and evaluation JSON hash. Absolute machine paths and credentials are excluded from the manuscript. Paper-generation scripts read existing records; they do not launch evaluation or touch the test dataset.

C. Additional Diagnostics and Reporting Plan

Layer reconstruction. On fixed calibration inputs X_ℓ from the dense parent, report $\|(W'_\ell - W_\ell)X_\ell\|_F^2$ with its explicit element-count normalization, alongside a relative error denominator when nonzero. This is the physical local error, distinct from the damped quadratic cost. State whether later-layer inputs are dense-parent or propagated sparse inputs; the current endpoint builder uses the dense parent. A separately measured end-to-end sparse forward pass addresses accumulated network effects.

Completed validation versus open evidence. The numerical audit and three-epoch smoke support finite geometry, positive-definite solves, preserved BN statistics, fixed radius, and resume integrity. The complete-run learning trajectory, final accuracy, native mask identity, and final calibration cost remain to be imported. Direct-Q and multiple matched seeds remain unmeasured. No significance test or mean-plus-standard-deviation claim is justified by a single exploratory run.

Interpreting potential outcomes. Improved sparse accuracy with unchanged dense accuracy would be consistent with improved compatibility, but still needs the history control. Better dense and sparse accuracy together could be explained partly by a better parent. Improved alignment without improved accuracy would show that support-energy association is insufficient. Worse performance than Direct-Q would weaken the proposed value of historical smoothing. These outcomes should all be retained, without changing the pruning endpoint or selecting a different historical reference after test evaluation.

Scope of novelty. The general perturbation form is covered by preconditioned SAM; historical energy allocation appears in GEAR-SAM; compression-aware optimization and OBS reconstruction have their own precedents. The proposed research contribution is the particular conditional-compensability history and its experimentally tested connection to an unchanged native compressor. Until the required comparisons exist, the draft presents this as a hypothesis and a reproducible implementation, not as a proven superior optimizer or a first use of second-order information in training.